Делегирующий конструктор

```
class Name {
    char* n = nullptr;
    Name(size_t size): n(new char[size + 1]) {
        n[size] = '\0';
    }
public:                                  //An initializer for a delegating constructor must appear alone
    Name() = default;
    Name(const char* n): Name(strlen(n)) {
        memcpy (this->n, n, strlen(n));
    }
    Name(const Name& other): Name(other.n) { }
    ~Name() {
        delete [] n;
    }
};
int main () {
    Name test("First");
    Name test1("Second");
    Name test2 = test1;
}
```

Конструктор перемещения

```
class Point {
private:
    double x = 0.0;
    double y = 0.0;
public:
    Point() = default;
    Point(double x, double y = 1.4): x(x), y(y) {
        std::cout<< "Constructor" <<std::endl;
    }
    Point(const Point& s): x(s.x), y(s.y) {
        std::cout<< "Copy Constructor" <<std::endl;
    }
    Point(Point &&s): x(s.x), y(s.y) {
        s.x = 0.0;
        s.y = 0.0;
        std::cout<< "Move Constructor" <<std::endl;
    }
};

Point g(Point p) {
    return p;
}

int main () {
   Point p5 = g(Point());
}
```

Конструктор перемещения

Обычно вызывается либо при явном указании std::move:

```
Name p("T");
Name p2 = std::move(p);
```

//p после этого будет невалиден

, либо когда компилятор не применяет оптимизацию и создаёт объект на основании временного:

```
Name func (Name p) {
    Name p1;
    std::cout << "Hi\n";
    return p;
}
int main (){
    Name p("T");
    Name p1 = func(p);
}
```

RVO. Return value optimization

Вот тут, например, будет оптимизация и никакого конструктора перемещения и копирования вызвано не будет (хотя создаётся временный объект и копируется в создающий):

```
Name p1 = Name("Name");
```

Как и тут:

```
Name func () {
    std::cout << "Hi\n";
    return Name("Hello");
}
int main (){
    Name p = func();
}
```

NRVO. Named RVO

Здесь тоже будет выполнена оптимизация:

```
Name func () {
    Name p1("Test");
    std::cout << "Hi\n";
    return p1;
}
int main (){
    Name p = func();
}
```

Здесь адрес переменной p будет неявно передан в функцию и сама переменная p будет связана с возвращаемой переменной из функции p1

Деструктор

- Деструктор – это метод класса, который вызывается перед уничтожением объекта. Для большинства типов он вообще ничего не делает. Но если есть нетривиальный конструктор, то и деструктор тоже должен быть нетривиальным.
- В остальных случаях он не нужен, даже объявлять его не надо.
- Деструкторы вызываются в порядке, обратном объявлению.
- Для каждого объекта деструктор должен быть вызван один раз.
- Деструктор можно вызывать явно.

Деструктор

```
class Name {
    char* n = nullptr;
public:
    Name(): n(new char){}
    Name(const char* n): n(new char[strlen(n) + 1]) {
        memcpy (this->n, n, strlen(n));
    }
    ~Name() {
        std::cout << "Dtor for: " << n <<std::endl;
        delete [] n;
    }
};
int main () {
    Name test("First");
    Name test1("Second");
    Name test2("Third");
}
```

Вывод:

```
Dtor for: Third
Dtor for: Second
Dtor for: First
Program ended with exit code: 0
```

Порядок вызова деструктора

```
class Surname {
    char* sur = nullptr;
public:
    Surname():sur(new char) {}
    Surname(const char* sur):sur(new char[strlen(sur)+1]){
        memcpy(this->sur,sur,strlen(sur));
        this->sur[strlen(sur)] = '\0';
    }
    ~Surname() {
        std::cout << "Dtor for surname: " << sur <<std::endl;
        delete [] sur;
    }
};

class Name {
    char* n = nullptr;
    Surname sur;
public:
    Name(): n(new char){}
    Name(const char* n, const char* sur): n(new char[strlen(n) + 1]), sur(sur) {
        memcpy (this->n, n, strlen(n));
    }
    ~Name() {
        std::cout << "Dtor for: " << n <<std::endl;
        delete [] n;
    }
};
```

```
int main () {
    Name test("First", "FirstSurname");
    Name test1("Second", "SecondSurname");
}
```

Вывод:

```
Dtor for: Second
Dtor for surname: SecondSurname
Dtor for: First
Dtor for surname: FirstSurname
Program ended with exit code: 0
```

Ключевое слово default

Когда тело стандартного метода может быть выбрано по умолчанию:

```
class Foo {
public:
    Foo() = default;
    Foo(int x) {/* ... */}
};
```

Ключевое слово default

Спецификатор **default** может применяться только к специальным функциям-членам:

- конструктор по умолчанию;
- конструктор копий;
- конструктор перемещения;
- оператор присваивания;
- оператор перемещения;
- деструктор.

Правило трёх

Если в классе присутствуют члены класса, которые требуют нетривиальной обработки, наличие конструктора копирования, деструктора и оператора присваивания обязательно! Даже если в данный момент не планируется дальнейшее их использование.

Правило пяти

Добавляются ещё перемещающие конструктор и оператор =

```
class Name {
    char* n = nullptr;
    Name(size_t size): n(new char[size + 1]) {
        n[size] = '\0';
    }
public:
    Name() = default;
    Name(const char* n): Name(strlen(n)) {
        std::cout << "Work\n" ;
        memcpy (this->n, n, strlen(n));
    }
    Name(const Name& other): Name(strlen(other.n)) {
        std::cout << "Work copy \n" ;
        memcpy (this->n, other.n, strlen(other.n));
    }
    Name& operator= (const Name& other) {
        if (this == &other)
            return *this;
        delete [] n;
        size_t length = strlen(other.n);
        n = new char[length + 1];
        memcpy (n, other.n, length);
        return *this;
    }
    ~Name() {
        delete [] n;
    }
};
```

Оператор присваивания, если отсутствует, тоже создаётся компилятором автоматически

Перемещающий оператор присваивания

```
Name& operator= (Name&& o) {
        if (this == &o) return *this;
        delete [] n;
        n = o.n;
        o.n = nullptr;
        return *this;
    }
```

Вызывается либо принудительно std:move

```
int main (){
    Name p = func();
    Name p1;
    p1 = std::move(p);
}
```

Либо при присваивании временного объекта:

```
int main (){
    Name p = func();
    p = Name{"Test"};
}
```

Идиома copy&swap

```
class Employee {
    char* name = nullptr;
    char* surname = nullptr;
    char date[11] {"01.01.1901"};
public:
    Employee () { }
    Employee (const char* n, const char *s, const char d[]):
          name(new char[strlen(n)+1])
        , surname(new char[strlen(s)+1]) {
            memcpy(name, n, strlen(n));
            memcpy(surname, s, strlen(s));
            memcpy(date, d, strlen(d));
    }
    Employee (const Employee& other): Employee(other.name, other.surname, other.date){}
    Employee& operator=(Employee other) { //const Employee& other) {
        //if(this ==&other) return *this;
        //Employee copy = other;
        swap(other); //swap(copy);
        return *this;
    }
    void swap(Employee& other) {
        std::swap(name, other.name);
        std::swap(surname, other.surname);
        std::swap(date, other.date);
    }
    ~Employee () {
        delete [] name;
        delete [] surname;
    }
};
```

Спецификатор delete

Спецификатор delete помечает те методы, работать с которыми нельзя.

```
class Name {
    char* n = nullptr;
    Name(size_t size): n(new char[size + 1]) {
        n[size] = '\0';
    }
public:
    Name() = default;
    Name(const char* n): Name(strlen(n)) {
        std::cout << "Work\n" ;
        memcpy (this->n, n, strlen(n));
    }
    Name(const Name& other) = delete;
    ~Name() {
        delete [] n;
    }

};

Name func (Name p) {
    std::cout << "Hi";
    return p;
}
```

Call to deleted constructor of 'Name'

Статические члены класса

В классе мы можем использовать поля и методы с ключевым словом static. Особенность их в том, что они принадлежат именно классам, а не конкретным экземплярам, то есть не тиражируются при создании объектов.

Статические переменные **должны инициализироваться вне классов**, вне зависимости от уровня доступа.

Пример

```
class Something {
private:
    static int s_value;
public:
    static int get() {
        return s_value;
    }
};


int Something::s_value{1};


int main() {
    std::cout << Something::get() << '\n';
    return 0;
}
```

const

```
class Name {
    char* n = nullptr;
    Name(size_t size): n(new char[size + 1]) {
        n[size] = '\0';
    }
public:
    char* getname() {
        return n;
    }
    Name() = default;
    Name(const char* n): Name(strlen(n)) {
        std::cout << "Work\n" ;
        memcpy (this->n, n, strlen(n));
    }
    Name(const Name& other): Name(strlen(other.n)) {
        std::cout << "Work copy \n" ;
        memcpy (this->n, other.getname(), strlen(other.n));   'this' argument to member function 'getname' has type 'const Name', but function is not marked const
    }
    Name& operator=(const Name& other) {
        if (this == &other)
            return *this;
        delete [] n;
        size_t length = strlen(other.n);
        n = new char[length + 1];
        memcpy (n, other.n, length);
        return *this;
    }
    ~Name() {
        delete [] n;
    }

};
```

```
class Name {
    char* n = nullptr;
    Name(size_t size): n(new char[size + 1]) {
        n[size] = '\0';
    }
public:
    char* getname() const {
        return n;
    }
    Name() = default;
    Name(const Name& other): Name(strlen(other.n)) {
        std::cout << "Work copy \n";
        memcpy (this->n, other.getname(), strlen(other.n));
    }
    Name& operator=(const Name& other) {
        if (this == &other)
            return *this;
        delete [] n;
        size_t length = strlen(other.n);
        n = new char[length + 1];
        memcpy (n, other.n, length);
        return *this;
    }
    ~Name() {
        delete [] n;
    }
};
```